Classifying Academic Subjects Based On QA texts

Mohd Tashwaruddin Safin
Computer Science and Engineering
Brac University
Dhaka, Bangladesh
mohd.tashwaruddin.safin@g.bracu.ac.bd

Fazle Mohammed Tasfiq
Department of Computer Science and Engineering
BRAC University
Dhaka, Bangladesh
fazle.mohammed.tasfiq@g.bracu.ac.bd

Angshu Nirmegh Biswas.

Department of Computer Science and Engineering
BRAC University
Dhaka, Bangladesh
angshu.niremgh.biswas@g.bracu.ac.bd

Abstract–Question-answer (QA) systems must not only retrieve relevant documents but also understand the semantic role of a query and the expected nature of its answer. Recognising whether a question is seeking a numeric fact, a location, a person's name or a descriptive explanation allows downstream QA components to narrow their search space and tailor their retrieval strategies. Deep learning models built on hierarchical recurrent or convolutional networks have improved the situation, but the use of contextualized representations like BERT and ELMo has yielded substantial further gains. The provided dataset contains a large training set and a hold-out test set of question–answer pairs labelled with answer types. Using the training set to learn robust feature representations and the test set for evaluation, we compare traditional machine-learning baselines with state-of-the-art RNN architecture, providing critical information for the discourse of classification in the field of Natural Language Processing.

Keywords—Classification, Machine Learning, Deep Neural Network, RNN, insert

Introduction

Natural language processing relies heavily on question answering for several applications, including personal assistants and automated customer care. Answer-type classification is a crucial aspect in QA pipelines. Given a natural-language query, it's necessary to choose the appropriate answer and interpret the user properly. Every week, industrial QA systems may process thousands of user inquiries, which can take many forms, including wh-questions, statement questions, and declarative statements. Identifying whether a response is a number, a location, an individual, an entity, or a descriptive explanation enables the retrieval engine to filter irrelevant passages and properly rank candidate options. Without classification, QA systems may provide irrelevant responses for straightforward inquiries, thus damaging the user experience.

Methodology

A. Dataset and Split

We use the provided QA-type classification corpus (balanced across classes; 280,003 samples). The original training set is used for model development and a held-out test set for final reporting. Within training, we reserve 10–20% for validation to select hyperparameters and perform early stopping.

B. Pre-processing

Text is normalized via lowercasing, punctuation stripping, stop-word removal, and lemmatization. Tokens are then produced for feature extraction. All steps are implemented in modular functions to ensure reproducibility and ease of reuse.

C. Feature Representations

BoW / TF-IDF: Unigram counts and TF-IDF weights form sparse vectors (with standard min-df and max-features limits to control dimensionality).

Distributional embeddings:

GloVe: Fixed, pre-trained embeddings are indexed by vocabulary.

Skip-gram Word2Vec: Custom embeddings are trained on the training text (window size and negative-sampling tuned).

These alternatives let us compare shallow ML baselines against neural models that consume dense, contextualized inputs.

D. Models

We benchmark two families:

Classical ML

Logistic Regression (multinomial) on BoW/TF-IDF

Multinomial Naive Bayes on BoW/TF-IDF.

Random Forest on BoW/TF-IDF.

Neural Networks

DNN on BoW/TF-IDF or averaged embeddings.

RNNs on embedded sequences:

SimpleRNN, GRU, LSTM and their bidirectional variants (e.g., Bi-GRU, Bi-LSTM). Architectures include an Embedding (pre-trained or trainable), recurrent layers (with dropout/spatial-dropout), optional dense layers, and a softmax output.

All implementations adhere to standard library defaults unless noted; hyperparameters (e.g., units per layer, dropout, learning rate) are tuned on validation.

05_22301047_21201160_22299019

E. Training Protocol

Optimizer & LR:

Adam with an initial learning rate of 1e-3; gradient-norm clipping (e.g., 1.0) to stabilize RNN training.

Batching & Epochs:

Mini-batches (e.g., 64–128) for up to 20–30 epochs with early stopping (patience 3–5).

Regularization:

Dropout on embeddings/recurrent outputs; L2 on dense weights as needed.

Initialization:

Pre-trained GloVe weights loaded into the embedding layer (frozen or fine-tuned). Skip-gram vectors are either averaged per sentence or fed token-wise to RNNs.

F. Evaluation

Primary metrics are accuracy and macro-F1, complemented by confusion matrices to inspect per-class errors. We report validation curves during development and final test performance only once, after model selection.

I. Ease of Use

The proposed experimental framework prioritizes reproducibility and practical applicability, ensuring ease of use for future researchers and practitioners. All preprocessing steps, including tokenization, stopword removal, and lemmatization, are systematically documented and implemented in modular functions, which allows for straightforward adaptation to various text datasets.

The incorporation of multiple word representation techniques—such as Bag of Words (BoW), Term Frequency-Inverse Document Frequency (TF-IDF), GloVe, and Skip-gram—alongside both traditional machine learning and advanced neural network models, establishes a comprehensive benchmark. This enables users to directly compare performance with minimal need for additional configuration. Hyperparameter tuning has been conducted transparently, with parameter ranges and justifications clearly articulated, facilitating both replication and further optimization.

From an implementation standpoint, the code is structured in a Jupyter Notebook format, which facilitates a comprehensive understanding through step-by-step explanations accompanying the code execution. This organization effectively lowers the entry barrier for new users, enabling even those without extensive expertise to follow along, make modifications, and build upon the experiments. The visualization of results—such as confusion matrices, classification reports, and comparative tables–further enhances interpretability and supports informed decision-making.

In summary, the project design is characterized by its modularity, transparency, reproducibility, and extensibility. This makes it not only suitable as a teaching resource but also a strong foundational tool for research and the development of applied text classification systems.

II. Paper information

A. Abbreviations and Acronyms

The following abbreviations and acronyms are used in this paper. Each is defined at its first occurrence in the main text.

- EDA: Exploratory Data Analysis
- NLP: Natural Language Processing
- BoW: Bag of Words
- TF-IDF: Term Frequency-Inverse Document Frequency
- GloVe: Global Vectors for Word Representation
- ML: Machine Learning
- NN: Neural Network
- LR: Logistic Regression
- NB: Naive Bayes
- RF: Random Forest
- DNN: Deep Neural Network
- RNN: Recurrent Neural Network
- SimpleRNN: A simple Recurrent Neural Network layer
- GRU: Gated Recurrent Unit
- LSTM: Long Short-Term Memory
- BiGRU: Bidirectional Gated Recurrent Unit
- BiLSTM: Bidirectional Long Short-Term Memory
- SI: International System of Units (Système International d'Unités)

B. Equations

This project employs a variety of mathematical models and functions. The key equations for the primary techniques are presented below.

- The Term Frequency-Inverse Document Frequency (TF-IDF) weighting scheme refines BoW by emphasizing informative terms. The weight of a term \(t \) in a document \(d \) from a corpus \(D \) is given by:

  wt,d=tft,dlog(dftN).

where \(\text{tf}_{t,d} \) is the term frequency of \(t \) in \(d \), \(N \) is the total number of documents, and \(\text{df}_t \) is the number of documents containing term \(t \).

- For the Machine Learning (ML) classifiers, the equations as follows. Logistic Regression (LR) models the probability of a class \(y \) given a feature vector \(x \) using the softmax function for multi-class classification:

  \[
  P(y = k | x) = \frac{e^{W_k \cdot x + b_k}}{\sum_{j} e^{W_j \cdot x + b_j}} \tag{3}
  \].

  where \(W_k \) and \(b_k \) are the weight vector and bias for class \(k \), respectively.

- The Multinomial Naive Bayes (NB) classifier applies Bayes' theorem with the "naive" assumption of feature independence. The predicted class is the one with the highest posterior probability:

  \[
  \hat{y} = \arg\max_k \left[\log P(y = k) + \sum_{i} \text{count}(w_i) \log P(w_i | y = k) \right]
  \].

- The Random Forest (RF) is an ensemble of \(K \) decision trees. The final prediction is made by majority voting:

  \[
  \hat{y} = \text{mode}\{ T_1(x), T_2(x), ..., T_K(x) \}
  \].

  where \(T_k(x) \) is the prediction of the \(k \)-th tree for sample \(x \).

- The Neural Network (NN) models utilize non-linear activation functions. The Rectified Linear Unit (ReLU) and hyperbolic tangent (tanh) functions were used in hidden layers:

  \[
  \text{ReLU}(z) = \max(0, z)
  \],

  \[
  \tanh(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}
  \].

- The logistic sigmoid function \(\sigma \), used in gating mechanisms for RNN) variants like the LSTM and GRU, is defined as:

  \[
  \sigma(z) = \frac{1}{1 + e^{-z}}
  \].

C. *Some Common Mistakes*

- The word "Data" is a plural noun. The training data are split into sets, not the data is split.
- Use "e.g." and "i.e." correctly. The abbreviation “e.g.” (from the Latin exempli gratia) means “for example,” and should be used when listing examples (e.g., classical ML models such as Logistic Regression, Naive Bayes, and Random Forest). The abbreviation “i.e.” (id est) means “that is,” and should be used to clarify or rephrase a statement (i.e., to provide a specific explanation).
- Distinguish "affect" and "effect." The choice of hyperparameters affects (verb, meaning to influence) the model's performance. The effect (noun, meaning the result) of using pre-trained embeddings was a significant accuracy boost.
- Differentiate between "compliment" and "complement." The BoW model can be enhanced or completed by the TF-IDF representation. The reviewer had nothing positive to say about the badly written report.
- Differentiate between "discrete" and "discreet." Discrete (adjective, meaning separate and distinct) numerical tokens are created from the text. All out-of-vocabulary words cannot be handled in a discrete (adjective, meaning cautious or unobtrusive) manner.
- Usually, the prefix "non" is joined without a hyphen. Nonlinear, nontrivial, and nonparametric are all accurate terms. Only in rare instances for clarity or when the root word is a proper noun (such as non-IEEE) is a hyphen used.
- quotation marks and punctuation. When referencing a complete element, periods and commas are enclosed in quote marks (e.g., the class was labeled "Answer."). However, punctuation is positioned outside of quotation marks when a single term is being emphasized (e.g., the model suffered from "vanishing gradients").
- Steer clear of ambiguous language. The imprecise word "essentially" should be replaced with "approximately" or "on average" (for example, the dataset roughly comprises 80,000 samples, not the dataset essentially comprises 80,000 samples).
- An "inset" is a graph inside a graph. The model's performance on the minority class is shown in Figure 5 as an inset rather than an insert.
- "That uses" as opposed to "using" If "that uses" is a suitable substitute for "using" in the paper title, capitalize the "u." For instance, "Multi-Class Classification Using Neural Networks" is accurate since it can also be interpreted as "Multi-Class Classification That Uses Neural Networks." The

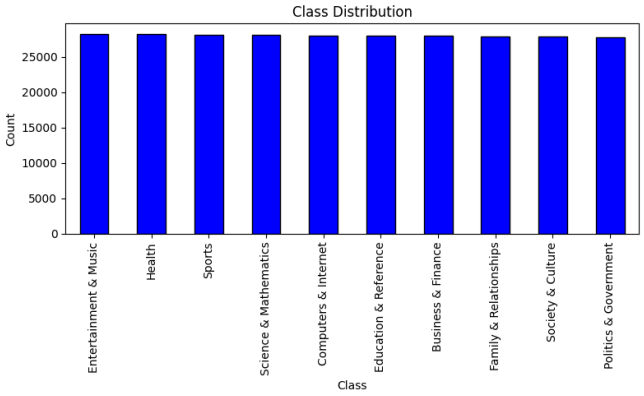
word "using" should not be capitalized because it is not a verb in this context.

III. EDA & Data Preprocessing

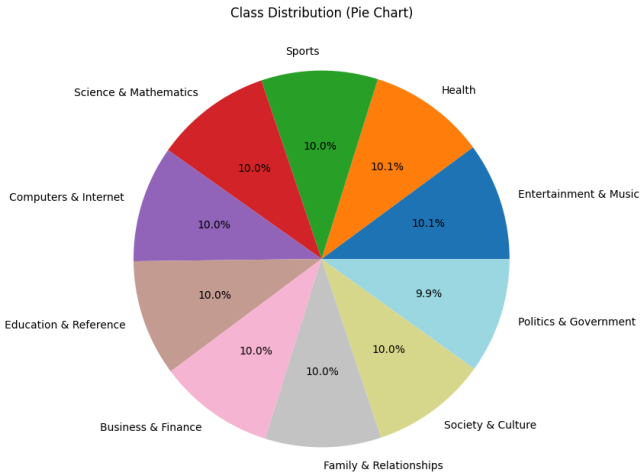
Here, the data preprocessing and EDA part has been shown–

A. EDA

The dataset has 280003 individual data, where all data are in object format. Here, all the classes has same amount of data. So, the dataset is well-balanced. Below the class distribution graph is giving -



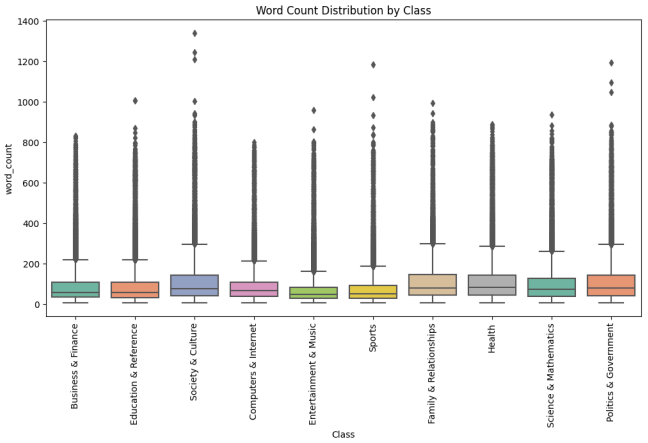
Also, the pie chart is given for better visualization of the class distribution percentage-



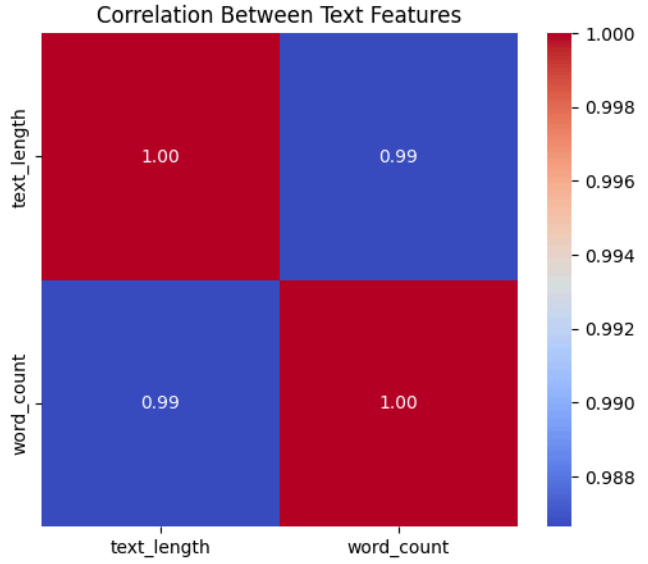
To know the most common word, we use wordcloud, where we get the most common words “Question, Content, Best etc.”-



Furthermore, we implement the text length function to know the most words length. Most of the words length is between 50 to 250 words. For getting better understanding, you can see the below plot-



Moreover, we check the correlation between the texts using heatmap-



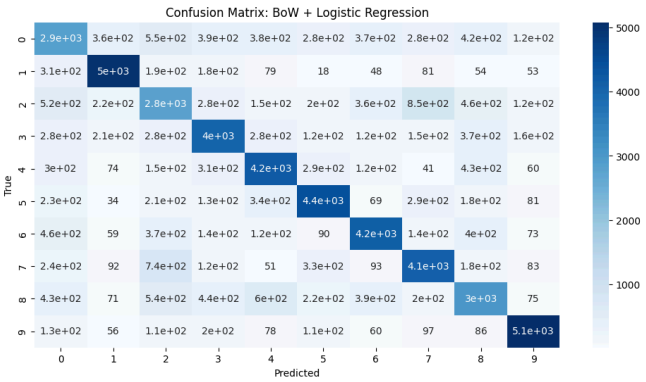
B. Data preprocessing

In data preprocessing, we clear the redundant words, tokenize the texts, remove the punctuations, and lemmatize the texts.

C. Figures and Tables

“Figure. 1” represents the confusion matrix for the model LogReg with BOW embedding.

FIGURE 1.



a. Sample

Fig. 1. Bow+LogReg Confusion Matrix

Figure Labels: The X-axis “ Fig. 1” represents the predicted label and Y-axis the True label

“Figure. 2” represents the confusion matrix for the model LogReg with TFidf embedding.

FIGURE 2.

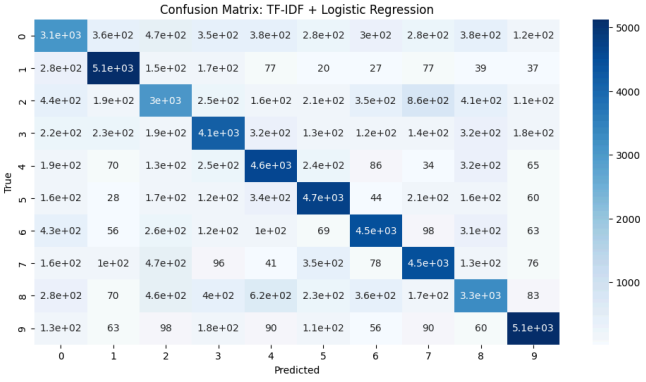


Fig.2. Tfidf+LogReg Confusion Matrix

Figure Labels: The X-axis “ Fig. 1” represents the predicted label and Y-axis the True label

FIGURE 3.

“Figure. 3” represents the confusion matrix for the model Navie Bayes with Bow embedding.

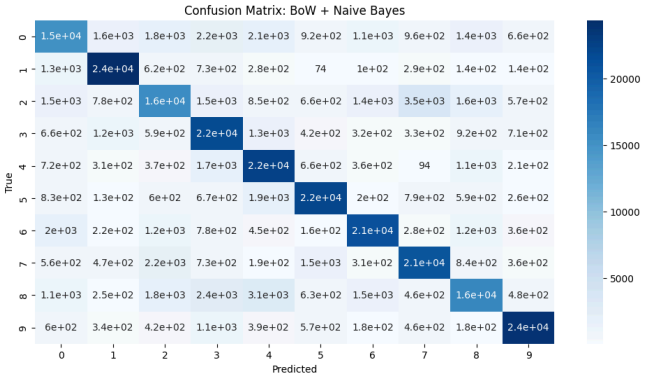


Fig.3. Tfidf+LogReg Confusion Matrix

Figure Labels: The X-axis “ Fig. 1” represents the predicted label and Y-axis the True label

FIGURE 4.

“Figure. 4” represents the confusion matrix for the model Navie Bayes with TfIdf embedding.

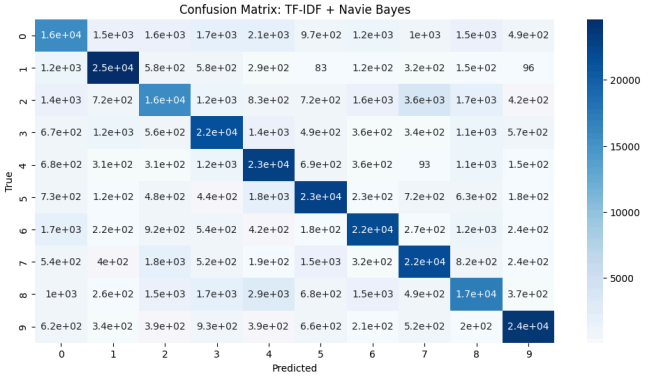


Fig.4. Tfidf+LogReg Confusion Matrix

Figure Labels: The X-axis “ Fig. 4” represents the predicted label and Y-axis the True label

FIGURE 5.

“Figure. 5” represents the confusion matrix for the DNN with Bow

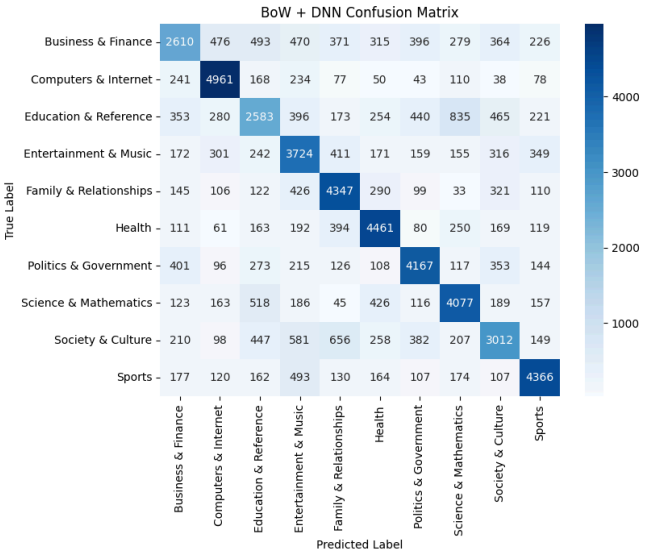


Fig.5. BOW+DNN Confusion Matrix

Figure Labels: The X-axis " Fig. 5" represents the predicted label and Y-axis the True label

"Figure. 6" represents the confusion matrix for the Model Vanilla BiRNN with Skipgram

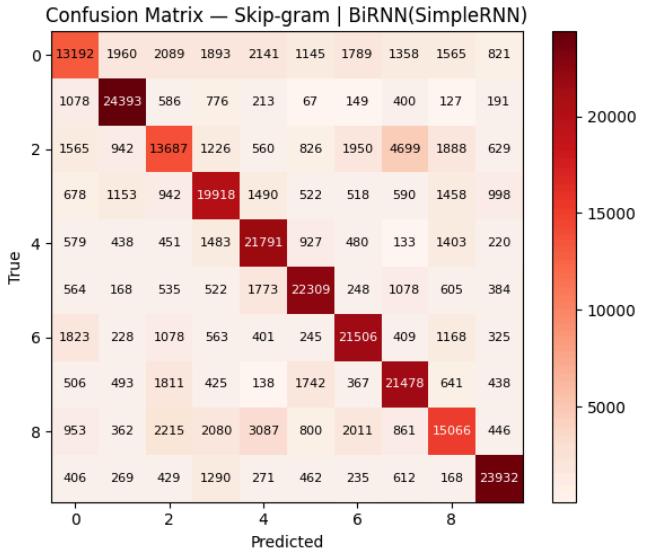
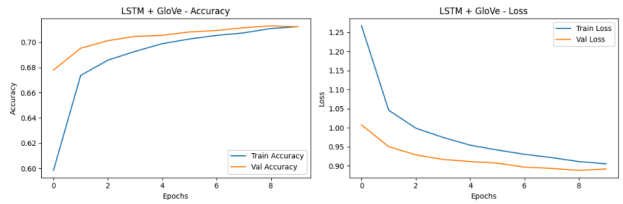


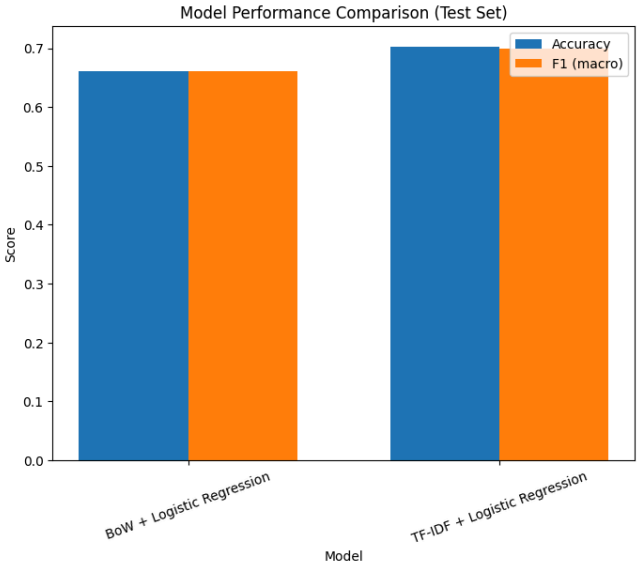
Fig.6. Skipgram+BiRNN

Figure:7 represents the LSTM + GloVe accuracy and loss.

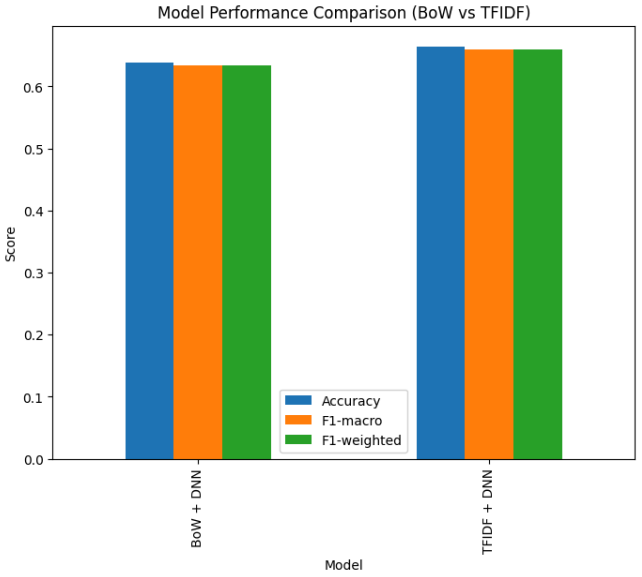


RESULTS.

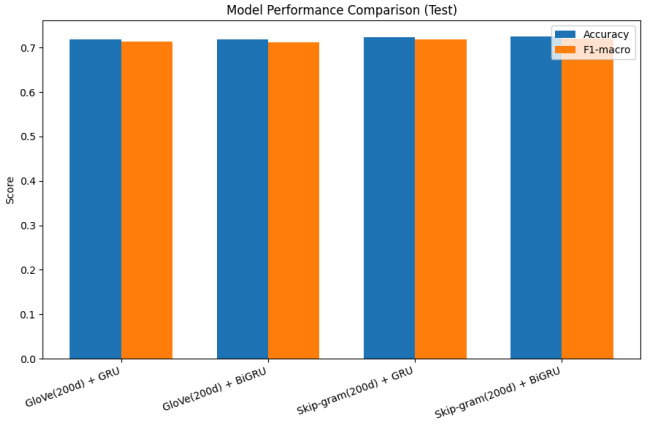
The performance of the models is varied because of different hyperparameters. The logistic regression give almost same performance on the test set for Bow and TF-IDF. Look at the below plot-



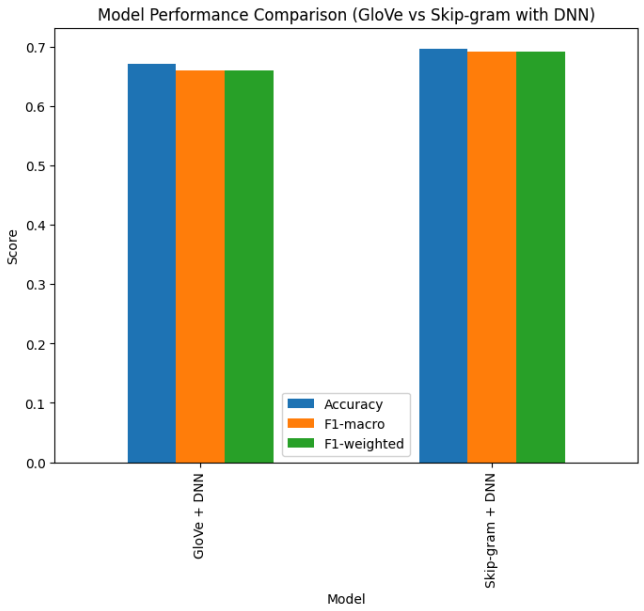
Then we implement the deep neural network for BoW and TF-IDF, which also gave almost close results.



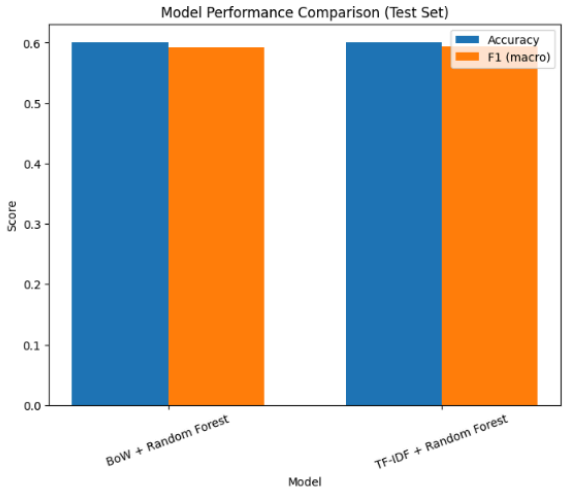
In a neural network, we implement the GRU and Bi-GRU using the Glove and Skipgram (trigram implemented). The performance of that model is-



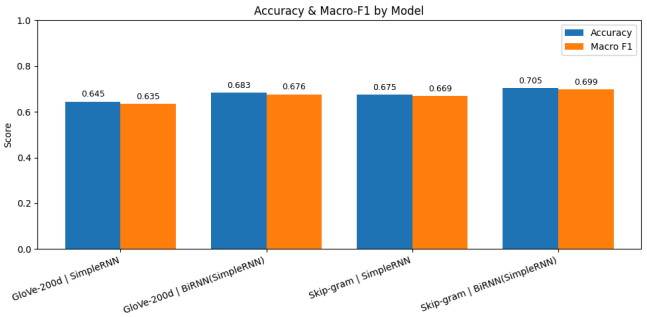
Then we implement the DNN model on the same GloVe and Skipgram, which drastically decreases the accuracy and F1-score,



We also implemented the Random forest for Bow and TF-IDF, which does not give good performance, as we see the below-



Simple/Vanilla RNN and Simple/Vanilla RNN performance and macro f1 performance over GloVE.200d and Skipgram.



We implemented more models like - Simple RNN and Bi-simple RNN, LSTM and Bi-LSTM, after all the model training, we find out that the Bi-GRU + Skipgram give the best result with the accuracy of 72.44 %.

Conclusion.

Trends across BoW/TF-IDF baselines and neural models are consistent with previous research: multinomial LR and NB give strong sparse-feature baselines; LogReg underperforms on very high-dimensional sparse text; and sequence models gain from dense embeddings. In our studies, Bi-GRU with Skip-gram embeddings produced the best overall result (72.44% accuracy), while DNNs with pre-trained embeddings failed to outperform the best recurrent arrangement. Confusion matrices reveal matching strengths across courses, indicating potential benefits from calibrated assembly. Further studies should investigate (i) domain-adaptive pre-training for embeddings, (ii) contextual encoders (e.g., transformer-based) with careful regularization on this dataset, and (iii) robust error analysis for minority phenomena.

References.

[1] scikit-learn TfidfVectorizer (usage & parameters). scikit

[2] scikit-learn LogisticRegression (multinomial, solvers, regularization). scikit-logreg

[3] scikit-learn MultinomialNB (when to use, alpha smoothing). scikit_NB

[4] Keras SimpleRNN layer API (arguments, examples). keras_VanilaRNN

[5] Keras GRU layer API (cuDNN notes, arguments). keras_GRU

[6] Keras Bidirectional wrapper (merge modes, usage). keras_bi